

JavaScript

```
// skills/bots/clawscript-parser.cjs  
// Expanded lexer/parser for ClawScript DSL  
// Now handles loops (LOOP/WHILE), try-catch error handling  
// Integrates OpenClaw AI tools by generating JS calls to wrappers (e.g.,  
require('../openclaw-ai.cjs').aiQuery(...))  
// Assume OpenClaw modules like openclaw-ai.cjs, openclaw-data.cjs, etc.,  
wrap tools (code_execution, browse_page, etc.)  
// Error handling: Parse-time throws, generate-time wraps in try-catch for  
runtime
```

```
const fs = require('fs');  
const path = require('path');
```

```
// Token types (unchanged)
```

```
const TOKEN_TYPES = {  
  KEYWORD: 'keyword',  
  IDENTIFIER: 'identifier',  
  STRING: 'string',  
  NUMBER: 'number',  
  OPERATOR: 'operator',  
  PUNCTUATION: 'punctuation',  
  COMMENT: 'comment'
```

```
};
```

```
// Lexer (unchanged)
```

```
function lexer(code) {  
  const tokens = [];  
  const regex = /\//.*|\\/.*[\\s\\S]*?\\*\\/|"(?:\\[^\\]|\\^\\")*"|\\d+\\.?.?\\d*[a-zA-Z_]*\\w*|=[><!&|+\\-/*%]+|\\(\\)\\{\\}\\[\\];,\\]/g;  
  let match;  
  while ((match = regex.exec(code))) {  
    const value = match[0];  
    if (value.startsWith('//') || value.startsWith('/*')) {  
      tokens.push({ type: TOKEN_TYPES.COMMENT, value });  
    } else if (value.startsWith('"')) {  
      tokens.push({ type: TOKEN_TYPES.STRING, value: value.slice(1, -1) });  
    } else if (/^\\d/.test(value)) {  
      tokens.push({ type: TOKEN_TYPES.NUMBER, value: parseFloat(value) });  
    } else if (/^[a-zA-Z_]/.test(value)) {  
      tokens.push({ type: TOKEN_TYPES.IDENTIFIER, value });  
    }
```

```

    } else if (/[=><!&|+\/\*%]/.test(value)) {
        tokens.push({ type: TOKEN_TYPES.OPERATOR, value });
    } else {
        tokens.push({ type: TOKEN_TYPES.PUNCTUATION, value });
    }
}
return tokens.filter(t => t.type !== TOKEN_TYPES.COMMENT);
}

// Parser class (expanded)
class ClawScriptParser {
    constructor(tokens) {
        this.tokens = tokens;
        this.pos = 0;
        this.ast = { type: 'Program', body: [] };
        this.variables = new Map();
        this.imports = new Set(['indicators']); // Default for trading
    }

    current() {
        return this.tokens[this.pos];
    }

    eat(type) {
        const token = this.current();
        if (token && token.type === type) {
            this.pos++;
            return token;
        }
        throw new Error(`Parse Error: Expected ${type}, got ${token ? token.type : 'EOF'}` at position ${this.pos}`);
    }

    parse() {
        while (this.pos < this.tokens.length) {
            try {
                const stmt = this.parseStatement();
                if (stmt) this.ast.body.push(stmt);
            } catch (err) {
                throw new Error(`${err.message} near token '${this.current()?.value}'`);
            }
        }
    }
}

```

```

    }
    return this.ast;
}

parseStatement() {
    const token = this.current();
    if (!token) return null;

    const upperValue = token.value.toUpperCase();
    switch (upperValue) {
        case 'DEF':
        case 'SET':
            return this.parseVarDeclaration(upperValue === 'DEF');
        case 'BUY':
        case 'SELL':
        case 'SELLSHORT':
            return this.parseTradeCommand(upperValue);
        case 'EXIT':
        case 'CLOSE':
            return this.parseExitCommand();
        case 'TRAILSTOP':
            return this.parseTrailStop();
        case 'IF':
            return this.parseIfStatement();
        case 'LOOP':
        case 'WHILE':
            return this.parseLoop(upperValue);
        case 'AI_QUERY':
            return this.parseAIQuery();
        case 'AI_GENERATE_SCRIPT':
            return this.parseAIGenerate();
        case 'ANALYZE_LOG':
            return this.parseAnalyzeLog();
        case 'RUN_ML':
            return this.parseRunML();
        case 'CLAW_WEB':
            return this.parseClawWeb();
        case 'CLAW_X':
            return this.parseClawX();
        case 'CLAW_PDF':
            return this.parseClawPdf();
        case 'CLAW_IMAGE':
            return this.parseClawImage();
    }
}

```

```

    case 'CLAW_VIDEO':
        return this.parseClawVideo();
    case 'CLAW_IMAGE_VIEW':
        return this.parseClawImageView();
    case 'SPAWN_AGENT':
        return this.parseSpawnAgent();
    case 'CALL_SESSION':
        return this.parseCallSession();
    case 'MUTATE_CONFIG':
        return this.parseMutateConfig();
    case 'ALERT':
        return this.parseAlert();
    case 'STORE_VAR':
        return this.parseStoreVar();
    case 'LOAD_VAR':
        return this.parseLoadVar();
    case 'WAIT':
        return this.parseWait();
    case 'TRY':
        return this.parseTryCatch();
    case 'ERROR':
        return this.parseError();
    case 'INCLUDE':
        return this.parseInclude();
    case 'OPTIMIZE':
        return this.parseOptimize();
    case 'INDICATOR':
        return this.parseIndicator();
    default:
        if (token.type === TOKEN_TYPES.IDENTIFIER) {
            return this.parseAssignment();
        }
        throw new Error(`Unexpected token: ${token.value}`);
    }
}

```

```

// Expanded: parseLoop for LOOP and WHILE
parseLoop(loopType) {
    this.eat(TOKEN_TYPES.IDENTIFIER); // LOOP/WHILE
    let condition = null;
    if (loopType === 'WHILE') {
        condition = this.parseExpression();
    } else { // LOOP num TIMES

```

```

    const num = this.parseExpression();
    this.eat(TOKEN_TYPES.IDENTIFIER); // TIMES
    condition = { type: 'LoopCount', num }; // Special node for count
  }
  const body = [];
  while (this.current() && this.current().value.toUpperCase() !==
'ENDLOOP' && this.current().value.toUpperCase() !== 'ENDWHILE') {
    body.push(this.parseStatement());
  }
  this.eat(TOKEN_TYPES.IDENTIFIER); // ENDLOOP/ENDWHILE
  return { type: 'Loop', loopType, condition, body };
}

// Expanded: parseTryCatch for TRY...CATCH...ENDTRY
parseTryCatch() {
  this.eat(TOKEN_TYPES.IDENTIFIER); // TRY
  const tryBody = [];
  while (this.current() && this.current().value.toUpperCase() !==
'CATCH') {
    tryBody.push(this.parseStatement());
  }
  this.eat(TOKEN_TYPES.IDENTIFIER); // CATCH
  const catchVar = this.eat(TOKEN_TYPES.IDENTIFIER).value; // var name
for error
  const catchBody = [];
  while (this.current() && this.current().value.toUpperCase() !==
'ENDTRY') {
    catchBody.push(this.parseStatement());
  }
  this.eat(TOKEN_TYPES.IDENTIFIER); // ENDTRY
  return { type: 'TryCatch', tryBody, catchVar, catchBody };
}

// parseError for ERROR "message"
parseError() {
  this.eat(TOKEN_TYPES.IDENTIFIER); // ERROR
  const message = this.parseExpression(); // Usually string
  return { type: 'ErrorThrow', message };
}

// [Other parse* methods unchanged or as before]

// Expanded Code Generation with OpenCLaw Integration

```

```

// Expanded code generation with openclaw integration
generateJS(ast, strategyName) {
  let js = `const BaseStrategy = require('./base-strategy.cjs');\n`;
  this.imports.forEach(imp => js += `const { ${imp} } = require('../
openclaw-${imp}.cjs');\n`);
  js += `\nclass ${strategyName}Strategy extends BaseStrategy {\n`;
  js += '  constructor(config) { super(config); }\n\n';
  js += '  async evaluateEntry(ticks, context) {\n'; // Async for awaits
(AI, web, etc.)
  ast.body.forEach(stmt => js += this.generateStmt(stmt, '    '));
  js += '    return null;\n  }\n\n';
  js += '  // Similar for evaluateExit\n';
  js += '  getRequiredBufferSize() { /* ... */ }\n';
  js += '  getConfigSchema() { /* ... */ }\n';
  js += '  static get STRATEGY_TYPE() { return "custom-
${strategyName.toLowerCase()}"; }\n';
  js += '};\nmodule.exports = ${strategyName}Strategy;\n';
  return js;
}

generateStmt(stmt, indent = '') {
  switch (stmt.type) {
    case 'VarDecl':
      return `${indent}${stmt.isDef ? 'const' : 'let'} ${stmt.name} =
${this.generateExpr(stmt.value)};\n`;
    case 'Trade':
      return `${indent}if (${this.generateExpr(stmt.condition)})
{\n${indent}  return { signal: true, direction: "${stmt.command}", size:
${stmt.size}, stopDist: ${this.generateExpr(stmt.stop)}, limitDist:
${this.generateExpr(stmt.limit)}, reason: ${this.generateExpr(stmt.reason)}
};\n${indent}}\n`;
    case 'Loop':
      let loopJs = `${indent}`;
      if (stmt.loopType === 'WHILE') {
        loopJs += `while (${this.generateExpr(stmt.condition)}) {\n`;
      } else {
        loopJs += `for (let i = 0; i <
${this.generateExpr(stmt.condition.num)}; i++) {\n`;
      }
      stmt.body.forEach(s => loopJs += this.generateStmt(s, indent + '
'));
      loopJs += `${indent}}\n`;
      return loopJs;
  }
}

```

```

    case 'TryCatch':
      let tryJs = `${indent}try {\n`;
      stmt.tryBody.forEach(s => tryJs += this.generateStmt(s, indent + '
'));
      tryJs += `${indent}} catch (${stmt.catchVar}) {\n`;
      stmt.catchBody.forEach(s => tryJs += this.generateStmt(s, indent + '
'));
      tryJs += `${indent}}\n`;
      return tryJs;
    case 'ErrorThrow':
      return `${indent}throw new
Error(${this.generateExpr(stmt.message)});\n`;
    case 'AIQuery':
      this.imports.add('ai');
      return `${indent}const ${stmt.var} || '_result' = await
ai.aiQuery("${stmt.prompt}", { tool: "${stmt.tool}", arg: "${stmt.arg}" });\n`;
// Integrates code_execution/browse_page/etc. via ai wrapper
// [Similar for other AI/data commands, e.g.]
    case 'ClawWeb':
      this.imports.add('data');
      return `${indent}const ${stmt.var} || '_result' = await
data.clawWeb("${stmt.url}", "${stmt.instruct}");\n`;
// Calls browse_page with instructions
    case 'ClawX':
      return `${indent}const ${stmt.var} || '_result' = await
data.clawX("${stmt.query}", { limit: ${stmt.limit}, mode: "${stmt.mode}"
});\n`;
// x_keyword_search
// [Expand for all: CLAW_PDF (search_pdf_attachment), RUN_ML (code_execution), SPAWN_AGENT (agent spawn via CEO), etc.]
    default:
      return `${indent}// Unsupported: ${stmt.type}\n`;
  }
}

// [generateExpr unchanged]
}

// Main (unchanged)
function parseAndGenerate(code, strategyName) {
  const tokens = lexer(code);
  const parser = new ClawScriptParser(tokens);
  const ast = parser.parse();
  return parser.generateJS(ast, strategyName);
}

```

```
}
```

```
module.exports = { parseAndGenerate, lexer, ClawScriptParser };
```